

Packet Capture and Protocol Analysis

Lab Report

Student Practical Laboratory Report

Course	CORE102 TCP IP Networking and Linux Network Tools
Instructor	Aminu Idris AMCPN
Maximum score	100 marks
Gradebook category	Practical Laboratories
Estimated duration	5 to 6 hours
Platform	Kali Linux or Ubuntu Linux in an authorised laboratory environment
Version	September 2026

Student name _MUHAMMAD HARUNA OSORI

Registration number _ C11/26/OCO/17007

Submission date _____

PART 1.

```
kali-linux-2026.2-virtualbox-amd64 (Snapshot 3) [Running] - Oracle VirtualBox : 1
File Machine View Input Devices Help

kali@kali: ~/core102_lab3

Session Actions Edit View Help
$ pwd
/home/kali/core102_lab3
(kali@kali)~[~/core102_lab3]
$ date
Fri Sep 25 06:42:17 AM EDT 2026
(kali@kali)~[~/core102_lab3]
$ hostname
kali
(kali@kali)~[~/core102_lab3]
$ ip -br addr
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.1.254/24 2605:59c0:e0d:d708:65dc:2719:9f27:57c4/64 fd9
eth1 UP 192.168.56.101/24 fe80::d9bd:df5b:c2a0:f2bd/64
(kali@kali)~[~/core102_lab3]
$ ip route
default via 192.168.1.1 dev eth0 proto dhcp src 192.168.1.254 metric 100
192.168.56.0/24 dev eth1 proto kernel scope link src 192.168.56.101 metric 101
(kali@kali)~[~/core102_lab3]
$ sudo tcpdump -D
[sudo] password for kali:
1.eth0 [Up, Running, Connected]
2.eth1 [Up, Running, Connected]
3.any (Pseudo-device that captures on all interfaces) [Up, Running]
4.lo [Up, Running, Loopback]
5.bluetooth-monitor (Bluetooth Linux Monitor) [Wireless]
6.nflog (Linux netfilter log (NFLOG) interface) [none]
7.nfqueue (Linux netfilter queue (NFQUEUE) interface) [none]
8.dbus-system (D-Bus system bus) [none]
9.dbus-session (D-Bus session bus) [none]
```

I created a directory for the practical “core102_lab3” and change directory to the working directotory. Then I run “hostname”, “date” and “ip -br addr” , “ip route” to record the system state.

The available to tcpdump when I run “sudo tcpdump -D” are; eth0, eth1, lo, bluthooth, and any (pseudo-device that capture on all interface). While the Following interface nflog, nfqueue, dbus-system, dbus-session display (none).

PART 2

```
kali@kali: ~/core102_lab3

Session Actions Edit View Help

(kali@kali)~[/core102_lab3]
$ ip route show default
default via 192.168.1.1 dev eth0 proto dhcp src 192.168.1.254 metric 102

(kali@kali)~[/core102_lab3]
$ NET_IFACE=$(ip route show default | awk 'NR==1 {print $5}')
GATEWAY=$(ip route show default | awk 'NR==1 {print $3}')
echo "Active Interface: $NET_IFACE"
echo "Gateway Address: $GATEWAY"

Active Interface: eth0
Gateway Address: 192.168.1.1

(kali@kali)~[/core102_lab3]
$ sudo tcpdump -ni "$NET_IFACE" -c 10 -s0 -w core102_icmp.pcap icmp
[sudo] password for kali:
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C8 packets captured
0 packets dropped by kernel

(kali@kali)~[/core102_lab3]
$ sudo tcpdump -nn -tttt -r core102_icmp.pcap
reading from file core102_icmp.pcap, link-type EN10MB (Ethernet), snapshot length 262144
2026-09-25 08:47:07.371392 IP 192.168.1.254 > 192.168.1.1: ICMP echo request, id 33888, seq 1,
length 64
2026-09-25 08:47:07.377206 IP 192.168.1.1 > 192.168.1.254: ICMP echo reply, id 33888, seq 1, l
ength 64
2026-09-25 08:47:08.405636 IP 192.168.1.254 > 192.168.1.1: ICMP echo request, id 33888, seq 2,
length 64
2026-09-25 08:47:08.430455 IP 192.168.1.1 > 192.168.1.254: ICMP echo reply, id 33888, seq 2, l
ength 64
2026-09-25 08:47:09.421731 IP 192.168.1.254 > 192.168.1.1: ICMP echo request, id 33888, seq 3,
length 64
2026-09-25 08:47:09.456038 IP 192.168.1.1 > 192.168.1.254: ICMP echo reply, id 33888, seq 3, l
ength 64
2026-09-25 08:47:10.434129 IP 192.168.1.254 > 192.168.1.1: ICMP echo request, id 33888, seq 4,

(kali@kali)~[/core102_lab3]
$ ping -c 4 "$GATEWAY"
ping: : Name or service not known

(kali@kali)~[/core102_lab3]
$ ping -c 4 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=13.3 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=24.9 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=34.4 ms
64 bytes from 192.168.1.1: icmp_seq=4 ttl=64 time=1.40 ms

— 192.168.1.1 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3069ms
rtt min/avg/max/mdev = 1.395/18.493/34.350/12.366 ms

(kali@kali)~[/core102_lab3]
$
```

Interface: eth0

Kali IP: 192.168.1.254

Gateway: 192.168.1.1

Capture file: core102_icmp.pcap

Procedure:

I identified the active interface and default gateway using “ip route “ before I Store the active interface name in NET_IFACE and the default gateway in GATEWAY . I started ICMP capture on eth0 using tcpdump. Four ICMP echo requests were then sent to the default gateway using “ping” the default gateway ip because the name GATEWAY is not sending ping .

Observation:

The capture showed ICMP Echo Request packets travelling from 192.168.1.254 to 192.168.1.1 and corresponding Echo Reply packets travelling back to 192.168.1.254. The sequence numbers increased, showing four request/reply pairs.

Result:

The ping completed successfully with **4 packets transmitted, 4 received and 0% packet loss**. This confirms successful ICMP communication between the Kali host and its default gateway.

Evidence: The saved PCAP file was “core102_icmp.pcap”. **which is attached to the zip folder included in the submission file**

Frame	Source	Destination	ICMP type	Identifier or sequence	Meaning
1	192.168.1.254	192.168.1.1	Echo Request	1	Kali asks router for a reply
1	192.168.1.1	192.168.1.254	Echo Reply	1	Router responds
2	192.168.1.254	192.168.1.1	Echo Request	2	Kali asks again
2	192.168.1.1	192.168.1.254	Echo Reply	2	Router responds
3	192.168.1.254	192.168.1.1	Echo Request	3	Kali asks again
3	192.168.1.1	192.168.1.254	Echo Reply	3	Router responds
4	192.168.1.254	192.168.1.1	Echo Request	4	Kali asks again
4	192.168.1.1	192.168.1.254	Echo Reply	4	Router responds
					Kali asks again

PART 3

```
Session Actions Edit View Help

(kali@kali)-[~/core102_lab3]
$ sudo tcpdump -ni eth0 -c 20 -s0 -w core102_dns.pcap '(udp port 53 or tcp port 53)'
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 by
tes
C4 packets captured
4 packets received by filter
0 packets dropped by kernel

(kali@kali)-[~/core102_lab3]
$ sudo tcpdump -nn -vv -r core102_dns.pcap '(udp port 53 or tcp port 53)'
reading from file core102_dns.pcap, link-type EN10MB (Ethernet), snapshot length 2
62144
11:58:11.854189 IP (tos 0x0, ttl 64, id 39045, offset 0, flags [none], proto UDP (
17), length 68)
10.0.2.15.40127 > 192.168.1.1.53: [bad udp cksum 0xcdf9 → 0x2880!] 38665+ [1a
u] A? example.com. ar: . OPT UDPsize=1472 (40)
11:58:11.893865 IP (tos 0x0, ttl 64, id 59414, offset 0, flags [none], proto UDP (
17), length 100)
192.168.1.1.53 > 10.0.2.15.40127: [udp sum ok] 38665 q: A? example.com. 2/0/1
example.com. A 172.66.147.243, example.com. A 104.20.23.154 ar: . OPT UDPsize=512
(72)
11:58:17.589789 IP (tos 0x0, ttl 64, id 38644, offset 0, flags [none], proto UDP (
17), length 66)
10.0.2.15.52470 > 192.168.1.1.53: [bad udp cksum 0xcdf7 → 0x888d!] 30256+ [1a
u] MX? gmail.com. ar: . OPT UDPsize=1472 (38)
11:58:17.632185 IP (tos 0x0, ttl 64, id 10812, offset 0, flags [none], proto UDP (
17), length 189)
192.168.1.1.53 > 10.0.2.15.52470: [udp sum ok] 30256 q: MX? gmail.com. 5/0/1 g
mail.com. MX alt2.gmail-smtp-in.l.google.com. 20, gmail.com. MX gmail-smtp-in.l.go
ogle.com. 5, gmail.com. MX alt3.gmail-smtp-in.l.google.com. 30, gmail.com. MX alt1
.gmail-smtp-in.l.google.com. 10, gmail.com. MX alt4.gmail-smtp-in.l.google.com. 40
ar: . OPT UDPsize=512 (161)

(kali@kali)-[~/core102_lab3]
$ dig gmail.com MX
; <<> DiG 9.20.27-2-Debian <<> gmail.com MX
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 39480
;; flags: qr rd ra; QUERY: 1, ANSWER: 5, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 65494
;; QUESTION SECTION:
;gmail.com.                IN      MX

;; ANSWER SECTION:
gmail.com.                630     IN      MX      20 alt2.gmail-smtp-in.l.go
.
gmail.com.                630     IN      MX      5 gmail-smtp-in.l.google.c
gmail.com.                630     IN      MX      30 alt3.gmail-smtp-in.l.go
.
gmail.com.                630     IN      MX      10 alt1.gmail-smtp-in.l.go
.
gmail.com.                630     IN      MX      40 alt4.gmail-smtp-in.l.go
.
.
;; Query time: 44 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Fri Sep 25 11:58:17 EDT 2026
;; MSG SIZE rcvd: 161

(kali@kali)-[~/core102_lab3]
$
```

The initial DNS test showed that the system was using the local resolver 127.0.0.53 rather than sending the DNS query directly through the active `eth0` interface. Therefore, the port-53 capture on `eth0` recorded no packets I have to troubleshoot it to get it capture traffic on `eth0` by going back to the VM configuration setting and change the network Adapter.

DNS traffic was successfully captured on `eth0`. An A query for `example.com` and an MX query for `gmail.com` were generated and observed. Both queries received successful responses from DNS server 192.168.1.1 on port 53. The client source ports, transaction IDs, response status, and returned records were identified from the packet capture. **4 packets were captured successfully.**

The capture recorded **4 DNS packets**: a request and response for each query. The client used temporary UDP source ports 40127 and 52470, while the DNS server listened on the standard DNS port 53.

The transaction IDs matched between each request and its corresponding response, confirming that the responses belonged to the correct queries.

Although `dig` displayed the local resolver `127.0.0.53`, the packet capture on `eth0` showed the DNS traffic being forwarded to `192.168.1.1`.

Query name	Type	Client port	Server	Response code	Answer summary
example.com	A	40127	192.168.1.1:53	NOERROR	172.66.147.243, 104.20.23.154
gmail.com	MX	52470	192.168.1.1:53	NOERROR	MX records including gmail-smtp- in.l.google.com and alt1-alt4.gmail- smtp- in.l.google.com

PART 4

```

kali@kali: ~/core102_lab3
Session Actions Edit View Help
(kali@kali)-[~/core102_lab3]
$ cd ~/core102_lab3
(kali@kali)-[~/core102_lab3]
$ echo '<h1>CORE102 authorised test page</h1>' > index.html
(kali@kali)-[~/core102_lab3]
$ python3 -m http.server 8080 --bind 127.0.0.1
Serving HTTP on 127.0.0.1 port 8080 (http://127.0.0.1:8080/)
127.0.0.1 - - [25/Sep/2026 12:09:11] "GET / HTTP/1.1" 200 -

(kali@kali)-[~/core102_lab3]
$ sudo tcpdump -ni lo -s0 -w core102_
http.pcap 'tcp port 8080'
[sudo] password for kali:
tcpdump: listening on lo, link-type EN1
0MB (Ethernet), snapshot length 262144
bytes
^C12 packets captured
24 packets received by filter
0 packets dropped by kernel

(kali@kali)-[~/core102_lab3]
$ sudo tcpdump -nn -A -r core102_http
.pcap 'tcp port 8080'
reading from file core102_http.pcap, li
nk-type EN10MB (Ethernet), snapshot len
gth 262144
12:09:11.137700 IP 127.0.0.1.55884 > 12
7.0.0.1.8080: Flags [S], seq 1933118026
, win 65495, options [mss 65495,sackOK,
TS val 2531135850 ecr 0,nop,wscale 9],
length 0
E..<..@.@.>.....L..s9
J.....0.....
...j.....
12:09:11.137720 IP 127.0.0.1.8080 > 127
.0.0.1.55884: Flags [S.], seq 168798278
, ack 1933118027, win 65483, options [m
ss 65495,sackOK,TS val 3620029168 ecr 2
531135850,nop,wscale 9], length 0
E..<..@.@.<.....L
..Fs9
K.....0.....

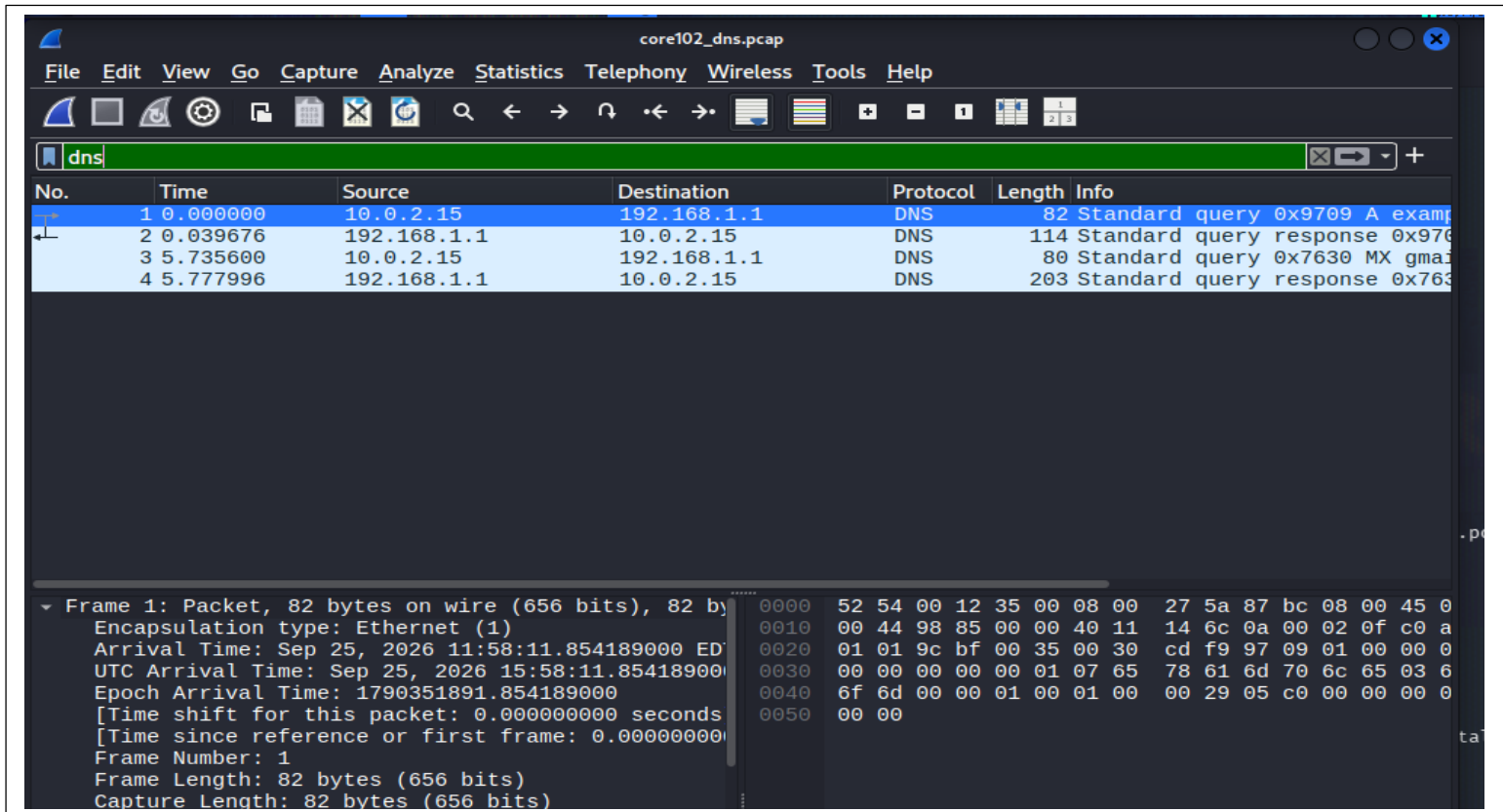
(kali@kali)-[~/core102_lab3]
$ curl -v http://127.0.0.1:8080/
* Trying 127.0.0.1:8080...
* Established connection to 127.0.0.1
127.0.0.1 port 8080) from 127.0.0.1
t 55884
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.1
< Date: Fri, 25 Sep 2026 16:09:11 G
< Content-type: text/html
< Content-Length: 38
< Last-Modified: Fri, 25 Sep 2026 1
:32 GMT
<
<h1>CORE102 authorised test page</h
* shutting down connection #0
(kali@kali)-[~/core102_lab3]
$

```

Stage	Display filter	Frame number	Evidence observed
Initial SYN	tcp.flags.syn == 1 && tcp.flags.ack == 0	1	127.0.0.1:55884 → 127.0.0.1:8080, Flags [S]
SYN ACK	tcp.flags.syn == 1 && tcp.flags.ack == 1	2	127.0.0.1:8080 → 127.0.0.1:55884, Flags [S.]
Final ACK	tcp.flags.ack == 1	3	127.0.0.1:55884 → 127.0.0.1:8080, Flags [.]
HTTP request	http.request	4	GET / HTTP/1.1
HTTP response	http.response	5	HTTP/1.0 200 OK and requested HTML content

I created three terminal , and I started python http server on the first terminal . create a tcpdump listing on the second terminal and the local HTTP traffic was successfully captured on the loopback interface. A Python web server was accessed using `curl`, and the capture showed the TCP handshake, HTTP GET request, successful 200 OK response, and connection closure. The test page was returned correctly, confirming that the HTTP communication was captured and analysed successfully.

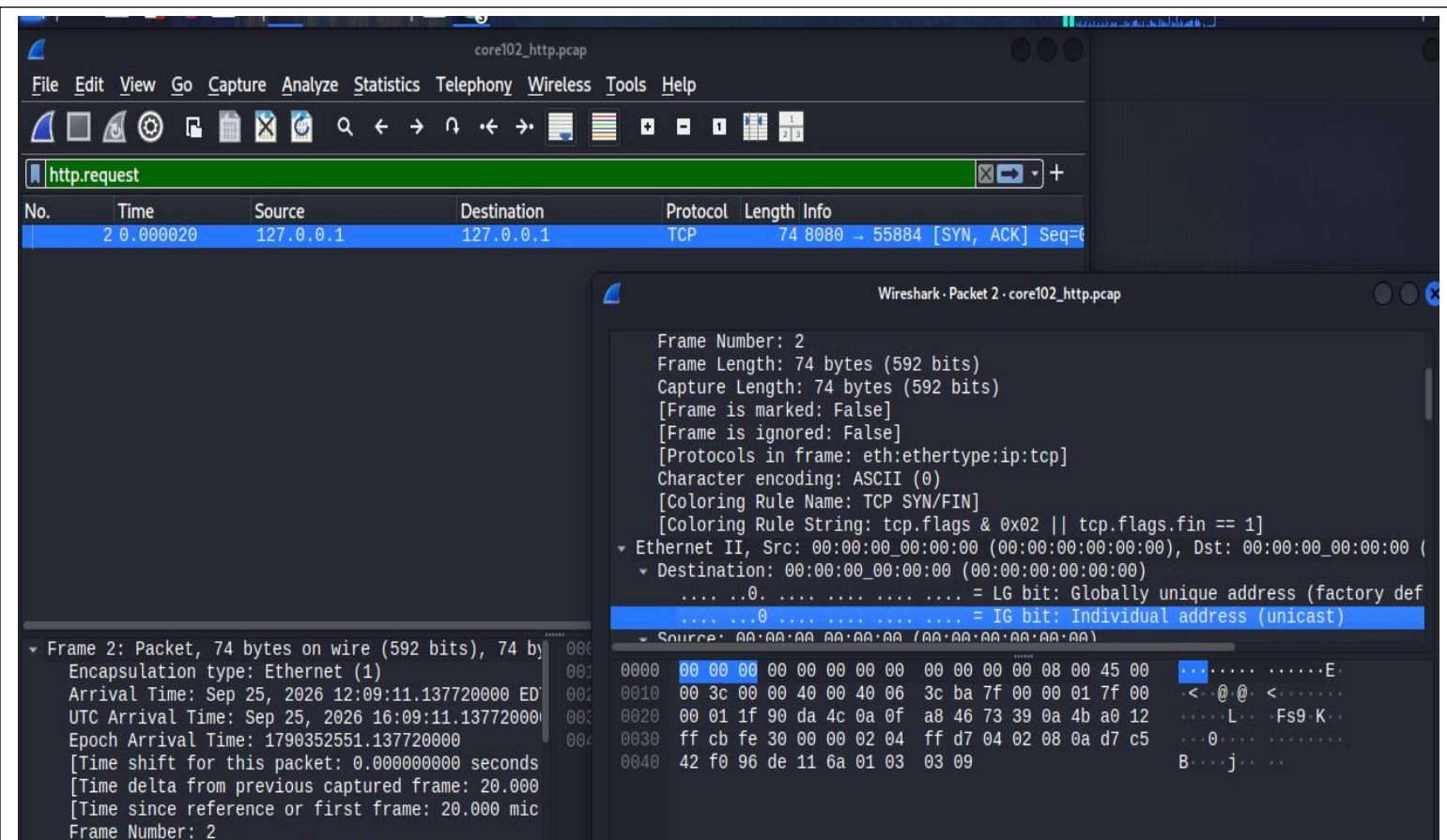
PART 5



The image shows a Wireshark capture of DNS traffic. The packet list pane displays four packets, with the first packet selected. The packet details pane shows the structure of the first packet, which is a DNS Standard query.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.2.15	192.168.1.1	DNS	82	Standard query 0x9709 A exam
2	0.039676	192.168.1.1	10.0.2.15	DNS	114	Standard query response 0x9709
3	5.735600	10.0.2.15	192.168.1.1	DNS	80	Standard query 0x7630 MX gma
4	5.777996	192.168.1.1	10.0.2.15	DNS	203	Standard query response 0x7630

Frame 1: Packet, 82 bytes on wire (656 bits), 82 bytes captured (656 bits) on interface eth0
Encapsulation type: Ethernet (1)
Arrival Time: Sep 25, 2026 11:58:11.854189000 EDT
UTC Arrival Time: Sep 25, 2026 15:58:11.854189000
Epoch Arrival Time: 1790351891.854189000
[Time shift for this packet: 0.000000000 seconds]
[Time since reference or first frame: 0.000000000 seconds]
Frame Number: 1
Frame Length: 82 bytes (656 bits)
Capture Length: 82 bytes (656 bits)



The image shows a Wireshark capture of HTTP traffic. The packet list pane displays two packets, with the first packet selected. The packet details pane shows the structure of the first packet, which is a TCP SYN, ACK.

No.	Time	Source	Destination	Protocol	Length	Info
2	0.000020	127.0.0.1	127.0.0.1	TCP	74	8080 → 55884 [SYN, ACK] Seq=

Frame 2: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface eth0
Encapsulation type: Ethernet (1)
Arrival Time: Sep 25, 2026 12:09:11.137720000 EDT
UTC Arrival Time: Sep 25, 2026 16:09:11.137720000
Epoch Arrival Time: 1790352551.137720000
[Time shift for this packet: 0.000000000 seconds]
[Time delta from previous captured frame: 20.000 seconds]
[Time since reference or first frame: 20.000 milliseconds]
Frame Number: 2

Wireshark interface showing a packet capture of ICMP Echo (ping) requests and replies. The filter is set to `icmp`.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.254	192.168.1.1	ICMP	98	Echo (ping) request id=0x846
2	0.005814	192.168.1.1	192.168.1.254	ICMP	98	Echo (ping) reply id=0x846
3	1.034244	192.168.1.254	192.168.1.1	ICMP	98	Echo (ping) request id=0x846
4	1.059063	192.168.1.1	192.168.1.254	ICMP	98	Echo (ping) reply id=0x846
5	2.050339	192.168.1.254	192.168.1.1	ICMP	98	Echo (ping) request id=0x846
6	2.084646	192.168.1.1	192.168.1.254	ICMP	98	Echo (ping) reply id=0x846
7	3.062737	192.168.1.254	192.168.1.1	ICMP	98	Echo (ping) request id=0x846
8	3.064069	192.168.1.1	192.168.1.254	ICMP	98	Echo (ping) reply id=0x846

Frame 1: Packet, 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface 0

Encapsulation type: Ethernet (1)

Arrival Time: Sep 25, 2026 08:47:07.371392000 EDT

UTC Arrival Time: Sep 25, 2026 12:47:07.371392000 UTC

Epoch Arrival Time: 1790340427.371392000

[Time shift for this packet: 0.000000000 seconds]

[Time since reference or first frame: 0.000000000 seconds]

Frame Number: 1

Frame Length: 98 bytes (784 bits)

Capture Length: 98 bytes (784 bits)

[Frame is marked: False]

[Frame is ignored: False]

[Protocols in frame: eth:ethertype:ip:icmp:data]

Character encoding: ASCII (A)

Wireshark interface showing a packet capture of HTTP traffic. The filter is set to `ip.addr eq 127.0.0.1 and ip.addr eq 127.0.0.1`.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	74	55884 → 8080 [SYN] Seq=0 Win=0
2	0.000020	127.0.0.1	127.0.0.1	TCP	74	8080 → 55884 [SYN, ACK] Seq=0
3	0.000035	127.0.0.1	127.0.0.1	TCP	66	55884 → 8080 [ACK] Seq=1 Ack=
4	0.000803	127.0.0.1	127.0.0.1	HTTP	144	GET / HTTP/1.1
5	0.000890	127.0.0.1	127.0.0.1	TCP	66	8080 → 55884 [ACK] Seq=1 Ack=
6	0.041503	127.0.0.1	127.0.0.1	TCP	252	8080 → 55884 [PSH, ACK] Seq=1
7	0.041542	127.0.0.1	127.0.0.1	TCP	66	55884 → 8080 [ACK] Seq=79 Ack=
8	0.041635	127.0.0.1	127.0.0.1	HTTP	104	HTTP/1.0 200 OK (text/html)
9	0.041643	127.0.0.1	127.0.0.1	TCP	66	55884 → 8080 [ACK] Seq=79 Ack=
10	0.041724	127.0.0.1	127.0.0.1	TCP	66	8080 → 55884 [FIN, ACK] Seq=2
11	0.042134	127.0.0.1	127.0.0.1	TCP	66	55884 → 8080 [FIN, ACK] Seq=7
12	0.042162	127.0.0.1	127.0.0.1	TCP	66	8080 → 55884 [ACK] Seq=226 Ac

Frame 8: Packet, 104 bytes on wire (832 bits), 104 bytes captured (832 bits) on interface 0

Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

Transmission Control Protocol, Src Port: 8080, Dst Port: 55884

[2 Reassembled TCP Segments (224 bytes): #6(186), #7(38)]

Hypertext Transfer Protocol

Line-based text data: text/html (1 lines)

core102_http.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp.port == 8080

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	74	55884 → 8080 [SYN] Seq=0 Win=
2	0.000020	127.0.0.1	127.0.0.1	TCP	74	8080 → 55884 [SYN, ACK] Seq=0
3	0.000035	127.0.0.1	127.0.0.1	TCP	66	55884 → 8080 [ACK] Seq=1 Ack=
4	0.000803	127.0.0.1	127.0.0.1	HTTP	144	GET / HTTP/1.1
5	0.000890	127.0.0.1	127.0.0.1	TCP	66	8080 → 55884 [ACK] Seq=1 Ack=
6	0.041503	127.0.0.1	127.0.0.1	TCP	252	8080 → 55884 [PSH, ACK] Seq=1
7	0.041542	127.0.0.1	127.0.0.1	TCP	66	55884 → 8080 [ACK] Seq=79 Ack=
8	0.041635	127.0.0.1	127.0.0.1	HTTP	104	HTTP/1.0 200 OK (text/html)
9	0.041643	127.0.0.1	127.0.0.1	TCP	66	55884 → 8080 [ACK] Seq=79 Ack=
10	0.041724	127.0.0.1	127.0.0.1	TCP	66	8080 → 55884 [FIN, ACK] Seq=2
11	0.042134	127.0.0.1	127.0.0.1	TCP	66	55884 → 8080 [FIN, ACK] Seq=7
12	0.042162	127.0.0.1	127.0.0.1	TCP	66	8080 → 55884 [ACK] Seq=226 Ack=

Frame 8: Packet, 104 bytes on wire (832 bits), 104 | 0000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 08 00 45 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00) | 0010 00 5a b9 ab 40 00 40 06 82 f0 7f 00 00 01 7f 0
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 | 0020 00 01 1f 90 da 4c 0a 0f a9 01 73 39 0a 99 80 1
Transmission Control Protocol, Src Port: 8080, Dst Port: 55884 | 0030 00 80 fe 4e 00 00 01 01 08 0a d7 c5 43 19 96 d
[2 Reassembled TCP Segments (224 bytes): #6(186), #7(38)] | 0040 11 93 3c 68 31 3e 43 4f 52 45 31 30 32 20 61 7
Hypertext Transfer Protocol | 0050 74 68 6f 72 69 73 65 64 20 74 65 73 74 20 70 6
Line-based text data: text/html (1 lines) | 0060 67 65 3c 2f 68 31 3e 0a

File Machine View Input Devices Help

core102_http.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp.flags.syn == 1 && tcp.flags.ack == 0

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	74	55884 → 8080 [SYN] Seq=0 Win=

Wireshark - Packet 1 - core102_http.pcap

Frame 1: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 0

Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

Transmission Control Protocol, Src Port: 55884, Dst Port: 8080, Seq: 0, Len: 0

Source Port: 55884

Destination Port: 8080

[Stream index: 0]

[Stream Packet Number: 1]

[Conversation completeness: Complete, WITH_DATA (31)]

[TCP Segment Len: 0]

Sequence Number: 0 (relative sequence number)

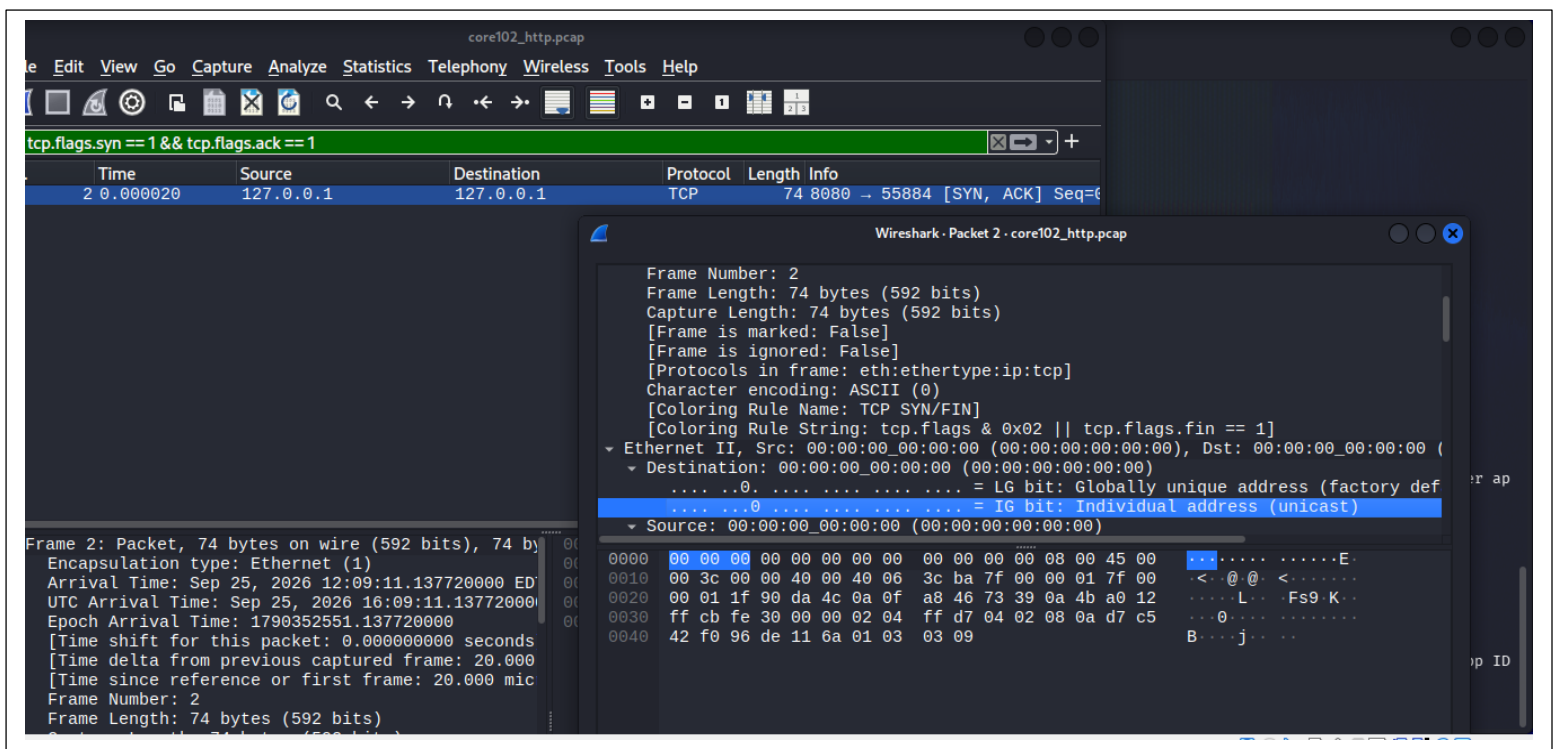
Sequence Number (raw): 1933118026

[Next Sequence Number: 1 (relative sequence number)]

Acknowledgment Number: 0

0000 00 00 00 00 00 00 00 00 00 00 00 00 00 08 00 45 00E
0010 00 3c fd c5 40 00 40 06 3e f4 7f 00 00 01 7f 00 00 <...@...>
0020 00 01 da 4c 1f 90 73 39 0a 4a 00 00 00 00 a0 02 00 ...L...s9.J.....
0030 ff d7 fe 30 00 00 02 04 ff d7 04 02 08 0a 96 de 00 ...0.....
0040 11 6a 00 00 00 00 01 03 03 09j.....

I were opened the captured PCAP files and analysed in Wireshark using the required display filters. The analysis identified DNS, ICMP and local HTTP traffic, as well as the TCP handshake and HTTP request. The TCP stream and protocol statistics were also reviewed to understand the communication flow and confirm that only authorised test traffic was present



PART 6

```
kali@kali: ~/core102_lab3
Session Actions Edit View Help
(kali@kali)~[~/core102_lab3]
$ cd ~/core102_lab3
(kali@kali)~[~/core102_lab3]
$ sudo tcpdump -ni eth1 -s0 -w core102_ssh.pcap 'tcp port 22'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 by tes
^C0 packets captured
0 packets received by filter
0 packets dropped by kernel
(kali@kali)~[~/core102_lab3]
$ sudo tcpdump -nn -vv -r core102_ssh.pcap 'tcp port 22'
reading from file core102_ssh.pcap, link-type EN10MB (Ethernet), snapshot length 262144
(kali@kali)~[~/core102_lab3]
$ cd ~/core102_lab3
sudo tcpdump -ni eth1 -s0 -w core102_ssh.pcap 'tcp port 22'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 by tes
^C1703 packets captured
1703 packets received by filter
0 packets dropped by kernel
(kali@kali)~[~/core102_lab3]
$
```

```
Usage of /: 13.3% of 24.44GB
Memory usage: 17%
Swap usage: 0%
Processes: 107
Users logged in: 0
IPv4 address for enp0s3: 192.168.1.163
IPv6 address for enp0s3: 2605:59c0:e0d:d708:a00:27ff:feb9:3d84
IPv6 address for enp0s3: fd9f:71da:400c:8:a00:27ff:feb9:3d84

* Canonical Workshop gives developers fast, composable, reproducible, and
  secure developer environments that are perfect for agentic workflows.

https://ubuntu.com/workshop

Expanded Security Maintenance for Applications is not enabled.

27 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Fri Sep 25 17:02:24 2026 from 192.168.1.254
ubuntu@ubuntu-server:~$ hostname
ubuntu-server
ubuntu@ubuntu-server:~$ whoami
ubuntu
ubuntu@ubuntu-server:~$ exit
logout
Connection to 192.168.56.102 closed.

(kali@kali)~[~/core102_lab3]
$
```

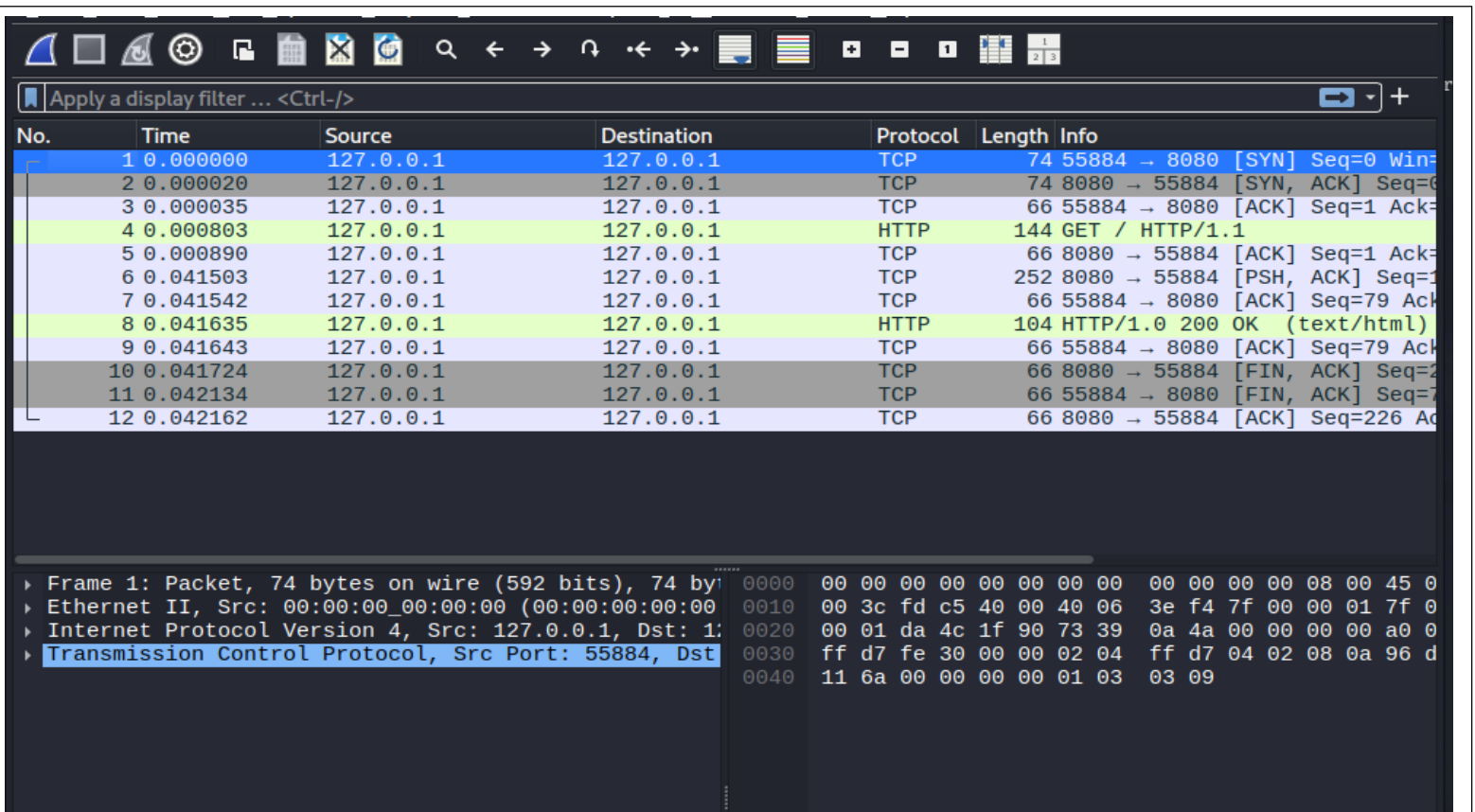
Capture reading

```
Session Actions Edit View Help
(kali@kali)~[~/core102_lab3]
$ sudo tcpdump -nn -vv -r core102_ssh.pcap 'tcp port 22'
reading from file core102_ssh.pcap, link-type EN10MB (Ethernet), snapshot length 262144
13:11:52.169040 IP (tos 0xb8, ttl 64, id 4282, offset 0, flags [DF], proto TCP (6), length 60)
  192.168.56.101.52440 > 192.168.56.102.22: Flags [S], cksum 0xf24a (incorrect -> 0x70a1), seq 3505931651, win 64240, options [mss 1460,sackOK,TS val 3316859185 ecr 0,nop,wscale 9], length 0
13:11:52.169472 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 60)
  192.168.56.102.22 > 192.168.56.101.52440: Flags [S.], cksum 0x67cb (correct), seq 2761542779, ack 3505931652, win 65160, options [mss 1460,sackOK,TS val 1320699231 ecr 3316859185,nop,wscale 8], length 0
13:11:52.169524 IP (tos 0xb8, ttl 64, id 4283, offset 0, flags [DF], proto TCP (6), length 52)
  192.168.56.101.52440 > 192.168.56.102.22: Flags [.], cksum 0xf242 (incorrect -> 0x94a2), seq 1, ack 1, win 126, options [nop,nop,TS val 3316859186 ecr 1320699231], length 0
13:11:52.169917 IP (tos 0xb8, ttl 64, id 4284, offset 0, flags [DF], proto TCP (6), length 85)
  192.168.56.101.52440 > 192.168.56.102.22: Flags [P.], cksum 0xf263 (incorrect -> 0xda34), seq 1:34, ack 1, win 126, options [nop,nop,TS val 3316859186 ecr 1320699231], length 33: SSH: SSH-2.0-OpenSSH_10.3p1 Debian-5
13:11:52.170362 IP (tos 0x0, ttl 64, id 49492, offset 0, flags [DF], proto TCP (6), length 52)
  192.168.56.102.22 > 192.168.56.101.52440: Flags [.], cksum 0x93ff (correct), seq 1, ack 34, win 255, options [nop,nop,TS val 1320699233 ecr 3316859186], length 0
13:11:52.188546 IP (tos 0xb8, ttl 64, id 49493, offset 0, flags [DF], proto TCP (6), length 94)
  192.168.56.102.22 > 192.168.56.101.52440: Flags [P.], cksum 0x5b7b (correct), seq 1:43, ack 34, win 255, options [nop,nop,TS val 1320699251 ecr 3316859186], length 42: SSH: SSH-2.0-OpenSSH_10.2p1 Ubuntu-2ubuntu3.6
13:11:52.188598 IP (tos 0xb8, ttl 64, id 4285, offset 0, flags [DF], proto TCP (6), length 52)
  192.168.56.101.52440 > 192.168.56.102.22: Flags [.], cksum 0xf242 (incorrect -> 0x9431), seq 34, ack 43, win 126, options [nop,nop,TS val 3316859205 ecr 1320699251], length 0
13:11:52.189087 IP (tos 0xb8, ttl 64, id 4286, offset 0, flags [DF], proto TCP (6), length 1724)
  192.168.56.101.52440 > 192.168.56.102.22: Flags [P.], cksum 0xf8ca (incorrect -> 0x0674), seq 34:1706, ack 43, win 126, options [nop,nop,TS val 3316859205 ecr 1320699251], length 1672
13:11:52.194516 IP (tos 0xb8, ttl 64, id 4288, offset 0, flags [DF], proto TCP (6), length 276)
  192.168.56.101.52440 > 192.168.56.102.22: Flags [P.], cksum 0xf322 (incorrect -> 0x5b29), seq 1482:1706, ack 43, win 126, options [nop,nop,TS val 3316859211 ecr 1320699251], length 224
13:11:52.194899 IP (tos 0xb8, ttl 64, id 49494, offset 0, flags [DF], proto TCP (6), length 64)
  192.168.56.102.22 > 192.168.56.101.52440: Flags [.], cksum 0x1da5 (correct), seq 43, ack 1706, win 249, options [nop,nop,TS val 1320699257 ecr 3316859205,nop,nop,ack 1 {1482:1706}], length 0
13:11:52.199744 IP (tos 0xb8, ttl 64, id 49495, offset 0, flags [DF], proto TCP (6), length 1092)
  192.168.56.102.22 > 192.168.56.101.52440: Flags [P.], cksum 0x866e (correct), seq 43:1083, ack 1706, win 249, options [nop,nop,TS val 1320699262 ecr 3316859205],
```

The authorised SSH session to the second virtual machine was successfully established and monitored using `tcpdump` on my first terminal . The capture showed the TCP three-way handshake, followed by encrypted SSH payload packets and the connection closure. Although I execute commands `hostname` and `whoami` according to the lab manual during the session, their contents were not readable in the packet capture because SSH encrypts the session data. This confirmed that the SSH communication was successfully captured while protecting the contents of the session.

PART 7

Use at least one of your PCAP files to reconstruct a short timeline. Reference exact frame numbers and timestamps.



Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	74	55884 → 8080 [SYN] Seq=0 Win=0
2	0.000020	127.0.0.1	127.0.0.1	TCP	74	8080 → 55884 [SYN, ACK] Seq=0
3	0.000035	127.0.0.1	127.0.0.1	TCP	66	55884 → 8080 [ACK] Seq=1 Ack=
4	0.000803	127.0.0.1	127.0.0.1	HTTP	144	GET / HTTP/1.1
5	0.000890	127.0.0.1	127.0.0.1	TCP	66	8080 → 55884 [ACK] Seq=1 Ack=
6	0.041503	127.0.0.1	127.0.0.1	TCP	252	8080 → 55884 [PSH, ACK] Seq=1
7	0.041542	127.0.0.1	127.0.0.1	TCP	66	55884 → 8080 [ACK] Seq=79 Ack=
8	0.041635	127.0.0.1	127.0.0.1	HTTP	104	HTTP/1.0 200 OK (text/html)
9	0.041643	127.0.0.1	127.0.0.1	TCP	66	55884 → 8080 [ACK] Seq=79 Ack=
10	0.041724	127.0.0.1	127.0.0.1	TCP	66	8080 → 55884 [FIN, ACK] Seq=2
11	0.042134	127.0.0.1	127.0.0.1	TCP	66	55884 → 8080 [FIN, ACK] Seq=7
12	0.042162	127.0.0.1	127.0.0.1	TCP	66	8080 → 55884 [ACK] Seq=226 Ad

▶ Frame 1: Packet, 74 bytes on wire (592 bits), 74 by
 ▶ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00)
 ▶ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
 ▶ Transmission Control Protocol, Src Port: 55884, Dst Port: 8080

```

0000  00 00 00 00 00 00 00 00 00 00 00 00 08 00 45 00
0010  00 3c fd c5 40 00 40 06 3e f4 7f 00 00 01 7f 00
0020  00 01 da 4c 1f 90 73 39 0a 4a 00 00 00 00 a0 00
0030  ff d7 fe 30 00 00 02 04 ff d7 04 02 08 0a 96 d7
0040  11 6a 00 00 00 00 01 03 03 09
  
```

Time	Frame	Source	Destination	Protocol	Event and interpretation
0.000000	1	127.0.0.1	127.0.0.1	TCP	SEND REQUEST
0.000020	2	127.0.0.1	127.0.0.1	TCP	REPLY REQUEST
0.000035	3	127.0.0.1	127.0.0.1	TCP	SEND REQUEST
0.000803	4	127.0.0.1	127.0.0.1	HTTP	GET REQUEST
0.000890	5	127.0.0.1	127.0.0.1	TCP	SEND REQUEST
0.041635	8	127.0.0.1	127.0.0.1	HTTP	GET 200 OK

SO I USE core102_http.pcap

PART 6